

OnlyFriends

Integritetsbyrå

Abstract

OnlyFriends is a social media platform for staying connected with your real-life friends. It aims to be privacy friendly through e2e encryption and minimization of metadata leaks to third parties.

Contents

1. Terminology	1
2. Device Chains	1
2.1. Types	1
2.2. Adding devices	2
2.3. Synchronizing	2
2.4. Removing devices	3
2.5. Adding Friends	3
2.6. Removing Friends	4
2.7. Forks & Time Travel	4
2.8. State resolution	4
3. Networking	4
References	5

1. Terminology

- **App** An implementation of OnlyFriends.
- **User** An individual, described by a set of *device chains*.
- **Device** One app instance, belonging to one specific *user*.
- **Sibling** A *device* belonging to the same *user*.
- **Device Chain** A linked list of Entries, each pointing to an optional Body, expressing the state of a single *device*.
- **Entry** A cryptographically signed data type that is the building block of the *device graph*.

2. Device Chains

A device is described only by a device chain. The device chain is an append only linked list, where each entry is cryptographically signed using ed25519ph [1]. Everything a user does is included in the device chain: Every post, every comment, every friend.

2.1. Types

Entry is the root type of the device chain. Postcard is used for serialization.

```
type EntryId = Sha512<Entry>;
```

```
struct Entry {  
    /// ID of the previous [Entry`]  
    prev: EntryId,  
    /// Kind, ID, and length of the [Body`] of this entry, if any.  
    body: Option<(BodyContentKind, BodyId, u64)>,  
    /// Longest seen sibling device chain. Used as a vector clock.  
    siblings_len: Map<DeviceId, u64>,  
}
```

```
type BodyId = Sha512<Body>;
```

```

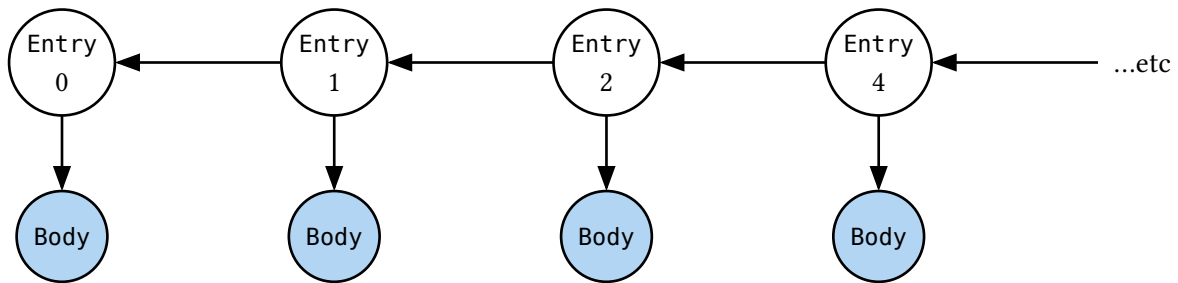
struct Body {
    content: BodyContent,
}

enum BodyContent {
    Post { /* ... */ }
    Reaction { /* ... */ }
    // etc
}

/// The Device ID is the Entry ID of the device chain root node.
type DeviceId = EntryId;

```

The ID types are SHA512 hashes of the identified value. Each Entry references its own Body by ID, as well as the previous Entry. This creates a linked list of entries – the device chain, which facilitates a common view from any given device. More details in Section 2.3.



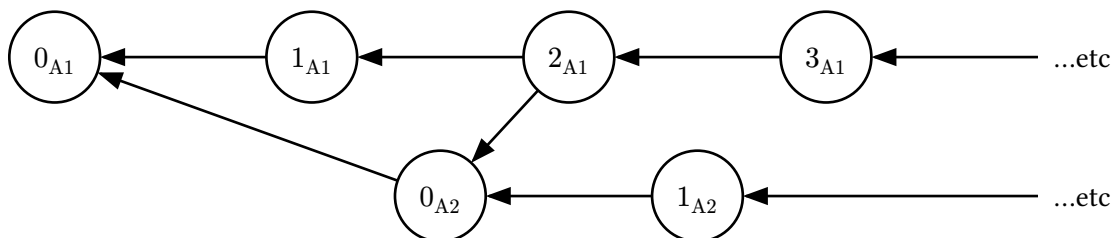
Bodys and Entrys serve two different purposes:

- An Entry is small, signed, with an upper bound on size. It's used to quickly synchronize the shape of a device chain. They also indicate Body *kind* and *length*.
- A Body can contain rich data, like user text and media. Data that may be large in size, and that a device may wish to discard or ignore to save on storage or network usage.

2.2. Adding devices

A new device A_2 is added by initializing its own device chain, vouching for A_1 and having A_1 vouch for it. A_1 and A_2 are siblings.

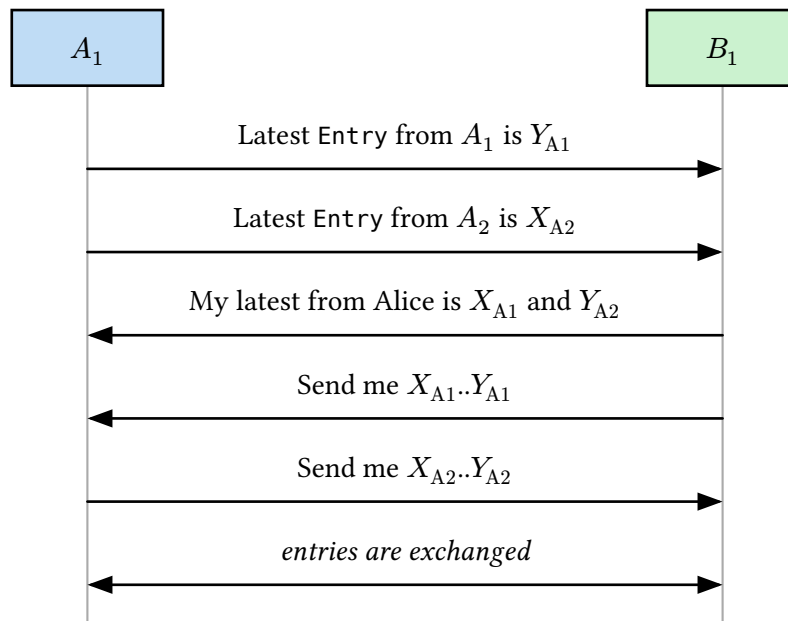
2_{A1} is an Entry from A_1 introducing A_2 as a sibling. This Entry is propagated to Alice's friends, allowing them to recognize A_2 . Likewise, having 0_{A2} vouch for A_1 's device chain, allows future friends introduced by A_2 to recognize the siblings.



2.3. Synchronizing

Let Alice and Bob be users. Let A_x be a device belonging to Alice, and B_x to Bob.

When Alice wants to synchronize with Bob, the message flow goes like this:



In this scenario, B_1 has seen a more recent Entry from A_2 than A_1 has. We allow a device (A_1) to synchronize Entries from any sibling (A_2) via any friend (Bob). Entries are small and cheap to send, and they are sent from oldest (X_{A1}) to newest (Y_{A1}), such that the signatures of each Entry can be verified immediately on reception.

After both devices are up to speed on Entries, They *may* ask each other to send Bodys as well.

Note that devices are not required to comply with synchronization requests. Apps may implement per-friend transfer quotas, to minimize the usefulness of exploiting friends for cheap storage.

2.4. Removing devices

Removing a device is mostly a formality. What it does, is to cap the devices' chain with a tombstone Entry, preventing other devices from accepting any new Entries from the removed device

There is, however, no harm in simply "forgetting" a device by locally deleting its private keys, but formally removing it allows other devices to present it as removed, improving UX.

A device can only be removed if it has an active connection to a sibling. Without a sibling, removal would likely be futile since a user might promptly delete the device app, leaving little opportunity for the tombstone message to be propagated to siblings in the normal fashion.

A removal is performed like so:

- The doomed device establishes a direct connection to a sibling device.
- The doomed device appends a tombstone message to its own device chain.
- The doomed device sends it to the sibling device, which is responsible for propagating the tombstone message to friends and other siblings.

A client must reject Entries of other devices for which it has seen a tombstone event, unless those Entries come before the tombstone message in the device chain.

TODO Should we allow a sibling to revoke a device without its cooperation? Would be nice if a device was, for example, stolen.

2.5. Adding Friends

A friend is added by a befriend-Entry. The Body is encrypted, and includes the device chain root (0th Entry and Body) of the friend.

When friends are added, two devices from different users exchange their device chain roots (and the root of the original device) over a secure side channel. The 0th Body contains a device's public keys, which facilitates secure communication over normal channels.

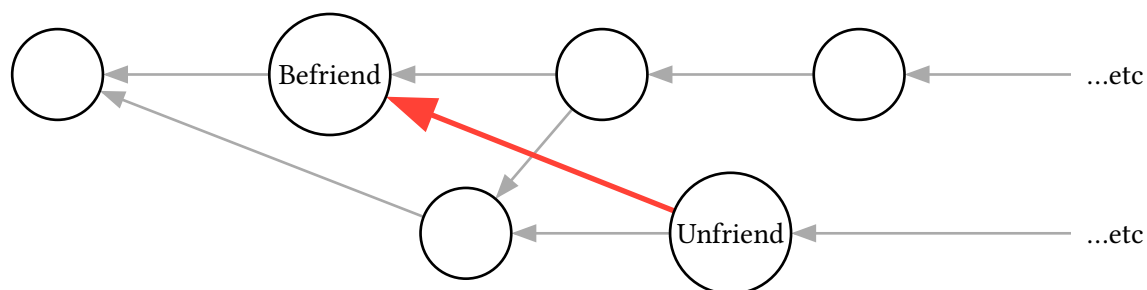
After the 0th is exchanged, synchronization happens as detailed in Section 2.3.

TODO With befriends/unfriends being present in the device chain, this means that your friends can know roughly how many friends you have. We can avoid this metadata leakage by maintaining a parallel encrypted device chain that we don't share with friends, but sync would be more fragile, since we couldn't bounce the device chain via friends. Presently, we leak this metadata:

- How many friends we have
- When any friend was added

2.6. Removing Friends

A friend is removed by an unfriend-Entry. Its Body is encrypted, and references the Entry which added the friend.



When a friend is removed, devices must cease all communication with friend devices.

TODO Inform friend that it has been removed? If we don't do this, a friend might try to connect with you forever in futility.

2.7. Forks & Time Travel

It is possible for malicious or malfunctioning devices to go back and create different, diverging, device chains (a fork). Protecting against this is not a priority, since there is little practical harm in a device attempting to rewrite its history, or trying to present different views to different friends.

Still, for the purposes of local consistency, a device must reject any incoming Entries that do not strictly append to a device chain. This way, forks are prioritized by their time of receipt, with the premise being this: Once a user has seen something (e.g. a post, a comment), it must not change.

Additionally, if a device observes a forked device chain for a friend, the app *may* choose to automatically generate an unfriend Entry, since the friend's device is either broken or malicious.

2.8. State resolution

Two siblings may independently change the same user state, creating a conflict. For example, both devices may update the user's profile picture. These conflicts are resolved by a vector clock[2], and if the clock is inconclusive, by the lowest DeviceId.

When a device receives a new sibling message, it must include the length of its sibling's device chain in the next Entry it appends to its own device chain. This creates the vector clock.

TODO Should the vector clock use the tail EntryId of the sibling chains instead of the length?

3. Networking

TODO

References

- [1] IRTF, “RFC 8032: Edwards-Curve Digital Signature Algorithm (EdDSA).” [Online]. Available: <https://www.rfc-editor.org/info/rfc8032/>
- [2] Wikipedia, “Vector clock.” [Online]. Available: https://en.wikipedia.org/wiki/Vector_clock